

Specification for Video Processing DSL

1. DSL Grammar Specification

1.1 Lexical Elements

TOKEN_TYPES:

IDENTIFIER: [a-zA-Z][a-zA-Z0-9_]*
STRING_LITERAL: "[^"]*"
INTEGER_LITERAL: [0-9]+
TIME_VALUE: [0-9]+(s|ms|:[0-9]{2}(:[0-9]{2})?)
COMMENT: #[^\n]*
PIPE_OPERATOR: \|>
PATH_LITERAL: "[^"]*\.(mp4|mp3)"
VARIABLE_REF: [a-zA-Z][a-zA-Z0-9_]*

KEYWORDS:

"open", "blank", "save", "show", "as", "trim", "overwrite",
"overlap"

1.2 Syntax Rules (EBNF Notation)

Program ::= Statement+

Statement ::= (Declaration | Command | Pipeline) "\n"

Declaration ::=
 StringDeclaration
 | StreamDeclaration

StringDeclaration ::= IDENTIFIER STRING_LITERAL

StreamDeclaration ::= "open" IDENTIFIER (STRING_LITERAL |
VARIABLE_REF)

Command ::=
 OpenCommand
 | BlankCommand
 | SaveCommand
 | ShowCommand
 | AsCommand
 | TrimCommand
 | OverwriteCommand
 | OverlapCommand

OpenCommand ::= "open" IDENTIFIER (STRING_LITERAL | VARIABLE_REF)

BlankCommand ::= "blank"

SaveCommand ::= "save" (STRING_LITERAL | VARIABLE_REF)

ShowCommand ::= "show"

AsCommand ::= "as" IDENTIFIER

TrimCommand ::= "trim" TIME_VALUE

OverwriteCommand ::= "overwrite" IDENTIFIER TIME_VALUE TIME_VALUE?

OverlapCommand ::= "overlap" IDENTIFIER TIME_VALUE

Pipeline ::= (IDENTIFIER | Command) (PIPE_OPERATOR Command)+

Comment ::= COMMENT

2. DSL Semantics

2.1 Semantic Domains

Domain Definitions:

String = sequence of characters

Path = valid filesystem path (String)

Stream = internal representation of video/audio tracks

Time = duration or position (seconds, milliseconds, or timestamp)

StreamVariable = reference to a Stream value

StringVariable = reference to a String value

2.2 Semantic Rules

2.2.1 Variables and Assignment

1. String Variables:

- String variables are mutable references to immutable string values
- Syntax: `identifier "value"`
- Example: `file2 "C:\Downloads\New video1.mp4"`

2. Stream Variables:

- Stream variables hold mutable stream values
- Created through commands or pipelines
- Syntax: `open identifier path` or `... |> as identifier`
- Example: `open vid1 "C:\Downloads\Video.mp4"`

2.2.2 Commands

- **open**: Creates a new stream from a file
 - Input: None or existing stream variable
 - Output: Stream
 - Parameters: Identifier (new variable name), Path (string literal or variable)
 - Semantics: Loads the media file at the specified path and creates a stream
- **blank**: Removes visual content from a stream
 - Input: Stream
 - Output: Modified stream with visual component removed
 - Parameters: None
 - Semantics: Transforms input stream by removing video tracks
- **save**: Persists a stream to a file
 - Input: Stream

- Output: Same stream (pass-through)
- Parameters: Path (where to save)
- Semantics: Writes the current stream state to the specified file location
- **show**: Displays stream in video player
 - Input: Stream
 - Output: Same stream (pass-through)
 - Parameters: None
 - Semantics: Sends the stream to the video player for display/preview
- **as**: Assigns stream to a variable
 - Input: Stream
 - Output: Same stream
 - Parameters: Identifier (variable name)
 - Semantics: Creates or updates a stream variable with the current stream
- **trim**: Cuts stream to specified duration
 - Input: Stream
 - Output: Modified stream
 - Parameters: Time value
 - Semantics: Truncates the stream to the specified duration
- **overwrite**: Replaces audio in target stream
 - Input: Stream (target)
 - Output: Modified stream
 - Parameters: Stream ID (source), start time, optional duration
 - Semantics: Replaces audio in target stream with audio from source stream
- **overlap**: Combines two streams
 - Input: Stream (target)
 - Output: Modified stream
 - Parameters: Stream ID (source), start time
 - Semantics: Overlays source stream on target stream at specified position

2.2.3 Pipelines

1. **Pipe Operator (|>):**
 - Chains commands together
 - Output of each command becomes input to the next
 - Semantics: Sequential execution of operations on streams

2.3 Error Conditions and Constraints

- **Type Errors:**
 - Using a string variable where a stream is expected
 - Using a stream variable where a string is expected
- **Reference Errors:**
 - Referencing undefined variables
 - Creating duplicate variable names
- **File Operation Errors:**
 - Invalid file paths
 - File not found
 - Permission errors
- **Time Value Errors:**
 - Negative time values
 - Time positions beyond stream duration
 - Invalid time format
- **Pipeline Errors:**
 - Commands with incompatible input/output requirements
 - Commands that require input used at beginning of pipeline

3. Grammar Complexity Analysis

- **Number of Productions:** Medium complexity
 - Approximately 20 distinct production rules
- **Recursion Depth:** Shallow recursion
 - Pipeline structures can nest operations but typically limited to 2-3 levels
- **Lexical and Syntactic Complexity:**
 - 8 token types (IDENTIFIER, STRING_LITERAL, etc.)
 - 8 keywords (open, blank, save, etc.)
 - Moderate nesting through pipeline chaining
 - Optional parameters in some commands
- **Ambiguity:** Mostly unambiguous
 - Clear parsing rules with limited alternative structures
 - Pipeline operator establishes clear precedence
 - Time value formats require disambiguation rules

4. Execution Model

The DSL follows a sequential execution model where:

1. Declarations establish variables in the environment
2. Commands operate on streams or produce new streams
3. Pipelines chain multiple operations in sequence
4. Variables provide named access to strings and streams
5. Side effects occur through save and show commands

5. Example Usage

```
# creates variable of type string
file2 "C:\Downloads\New video1.mp4"

#creates var vid2 using as path string variable file2
open vid2 file2

# creates var vid1 of type "stream" with dir specified,
# removed visualised, saved file as audio.mp3 in specified dir,
# then displayed in the video player,
# then saved as variable audio1 (also creates the variable)
open vid1 "C:\Downloads\Video.mp4" |> blank|> save ".../../audio.mp3"
|> show |> as audio1

# uses the already created variable vid1 to trim it by 5 sec,
# then saved as vid1 (overrights), then starts audio1,
# overwriting the existing audio (combination of mute and overlap)
# at sec 5 in vid1 and plays for 10 seconds,
# then starts vid2 and overlaps over vid1,
# playing vid2 visuals and both audios for the duration of overlap,
# until vid2 ends, then displays in the video player the result
# of the changes on vid1, then save the changes as vid6 variable,
# then save it as a file with name changed.mp4
vid1 |> trim 5s |> as vid1 |> overwrite audio1 00:05 10s|> operlap
vid2 01:00 |> show |> as vid6 |> save ".../../changed.mp4"
```